

# Computergrafik

## Vorlesung 2

Hochschule Zittau/Görlitz

Christopher-Manuel Hilgner



# Polygon Meshes

# Surfaces

Klassisch und allgegenwärtig in Computergrafik

- Erstellen polygonaler Objekte ist unkompliziert
- Es gibt visuell effiziente Algorithmen zur Erzeugung schattierter Bilder
- Eine Maschinendarstellung, konvertiert aus anderen Benutzerdarstellungen wie parametrische/impliziten Oberflächen, CSG/Volumen

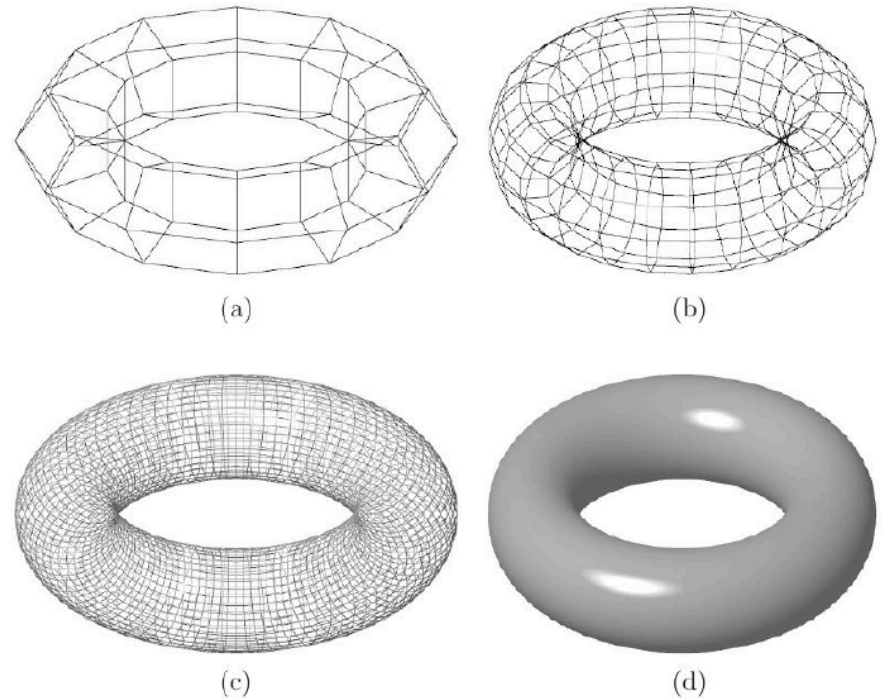
Schwierigkeiten bei Polygon Meshes:

- geometrische Genauigkeit
- Formmanipulation

# Polygon Meshes

Tori in verschiedenen  
Polygonauflösungsstufen:

- (a) 50 Quads
- (b) 200 Quads
- (c) 3200 Quads
- (d) 819200 Quads (da hier das Drahtgitter so fein ist, dass auf jedes Pixel mindestens eine Gitterlinie fällt, sieht es so aus, als ob der Torus eine geschlossene Oberfläche hätte)



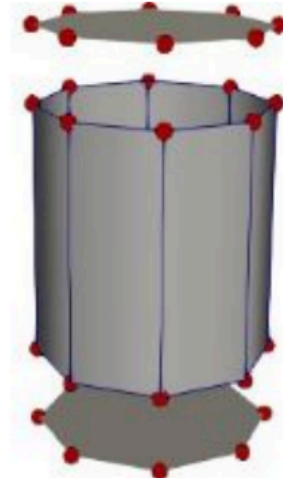
# Polygon Meshes



Objekt



Surface



Polygone

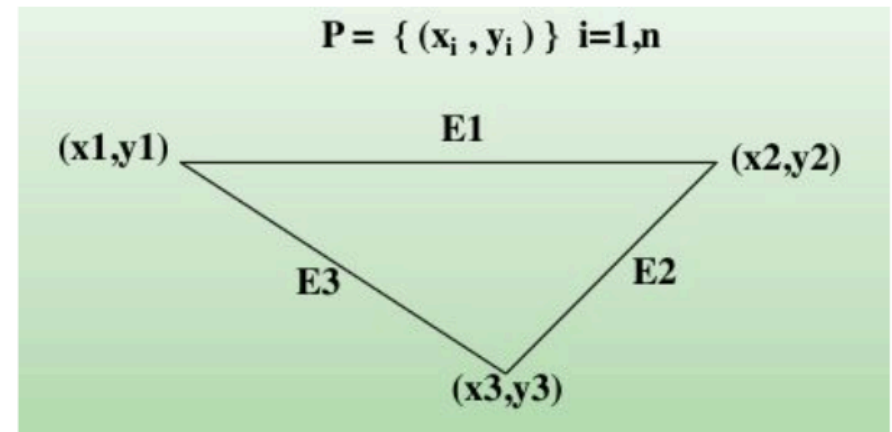


Edges/  
Vertices

# Polygon Meshes

## Definition:

- Ein Polygon ist eine vielseitige, ebene Figur, die aus Eckpunkten und Kanten besteht.
- Vertices werden durch Punkte  $(x,y)$  dargestellt ( $x,y,z$  in 3D)
- Edges/Kanten werden als Liniensegmente dargestellt, die zwei Punkte verbinden:  $(x1,y1)$  und  $(x2,y2)$



```
// Spezifikation von Vertices  
float vec[][3] = { 3.8, 0.47, -4.1,  
0.0, 0.71, -2.9,  
1.3, 5.33, -6.2 };
```

# Konvexe Polygone

- Konvexes Polygon - Bei zwei beliebigen Punkten P1, P2 innerhalb des Polygons befinden sich alle Punkte, die auf dem Liniensegment zwischen P1 und P2 liegen, innerhalb des Polygons.
- Alle Punkte

$$P = u \times P1 + (1 - u) \times P2, u \in [0, 1]$$

liegen innerhalb des Polygons, vorausgesetzt, dass P1 und P2 innerhalb des Polygons liegen

# Konkave Polygone

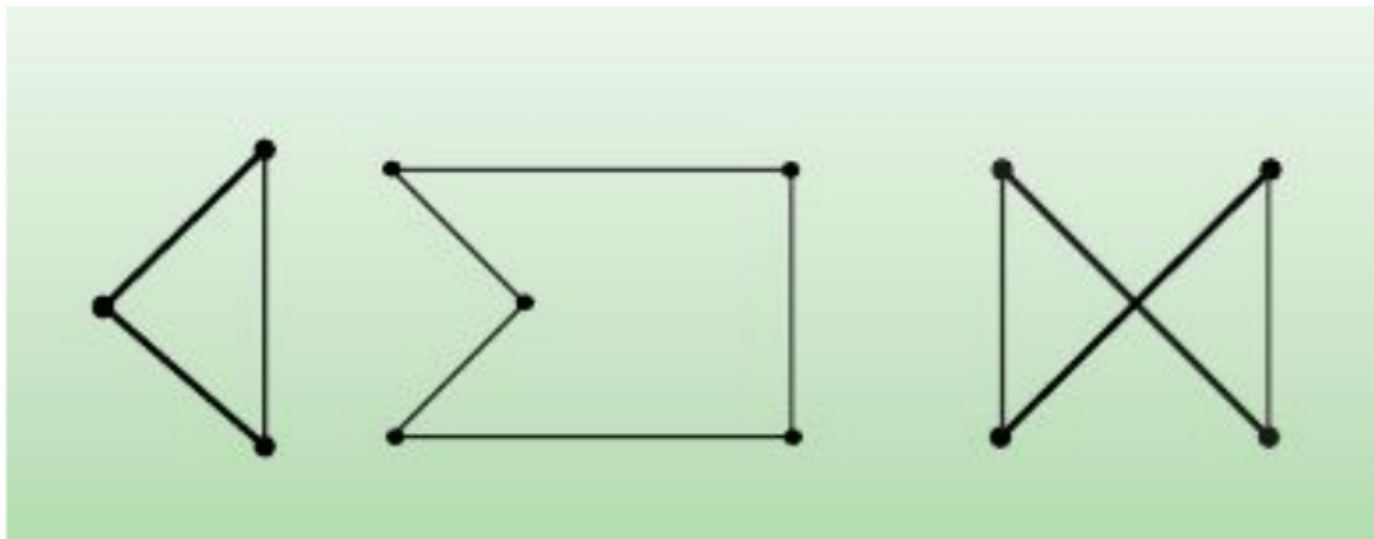
- Konkaves Polygon - Ein Polygon, das nicht konvex ist.

Well no shit, captain obvious!



# (Nicht) Einfache Polygone

- **Einfache Polygone** - Polygone, deren Kanten sich nicht kreuzen
- **Nicht einfache Polygone** - Polygone, deren Kanten sich kreuzen
  - Zwei verschiedene OpenGL-Implementierungen können nicht einfache Polygone unterschiedlich darstellen. OpenGL prüft nicht, ob Polygone einfach sind.
- Dreieck / Triangle
  - Polygon garantiert konvex und einfach (und planar in 3D)



# Mesh Representation

# Mesh Representation

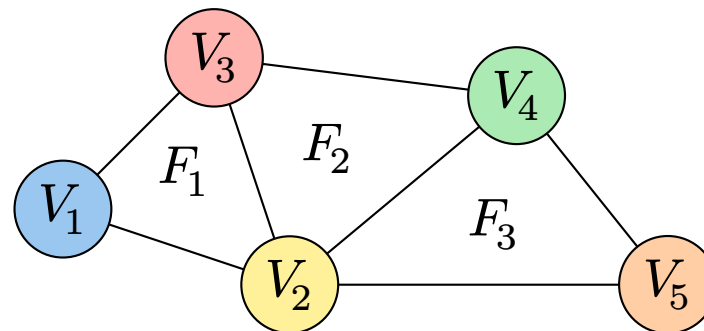
- Meshes können unterschiedlich abgespeichert und repräsentiert werden
- Mesh Representations:
  - Independent Faces
  - Vertex and Face Table
  - Adjacency Lists
  - (Winged Edge)

# Mesh Representation

- Independent Faces:
  - Jede Fläche listet Vertex-Koordinaten auf
    - Redundante Eckpunkte
    - Keine Topologie-Informationen
- Vertex und Face-Table
  - Jede Fläche listet Vertex-Referenzen auf
    - Gemeinsam genutzte Eckpunkte, kompaktere Darstellung
    - Immer noch keine Topologie-Informationen

# Mesh Representation

## Vertex und Face-Table



| Vertex Table |          |
|--------------|----------|
| v1           | x1 y1 z1 |
| v2           | x2 y2 z2 |
| v3           | x3 y3 z3 |
| v4           | x4 y4 z4 |
| v5           | x5 y5 z5 |

| Face Table |          |
|------------|----------|
| F1         | v1 v2 v3 |
| F2         | v2 v4 v3 |
| F3         | v2 v5 v4 |

# Mesh Representation

**Adjacency Lists:** Alle Vertex-, Kanten- und Flächenadjazenzen speichern

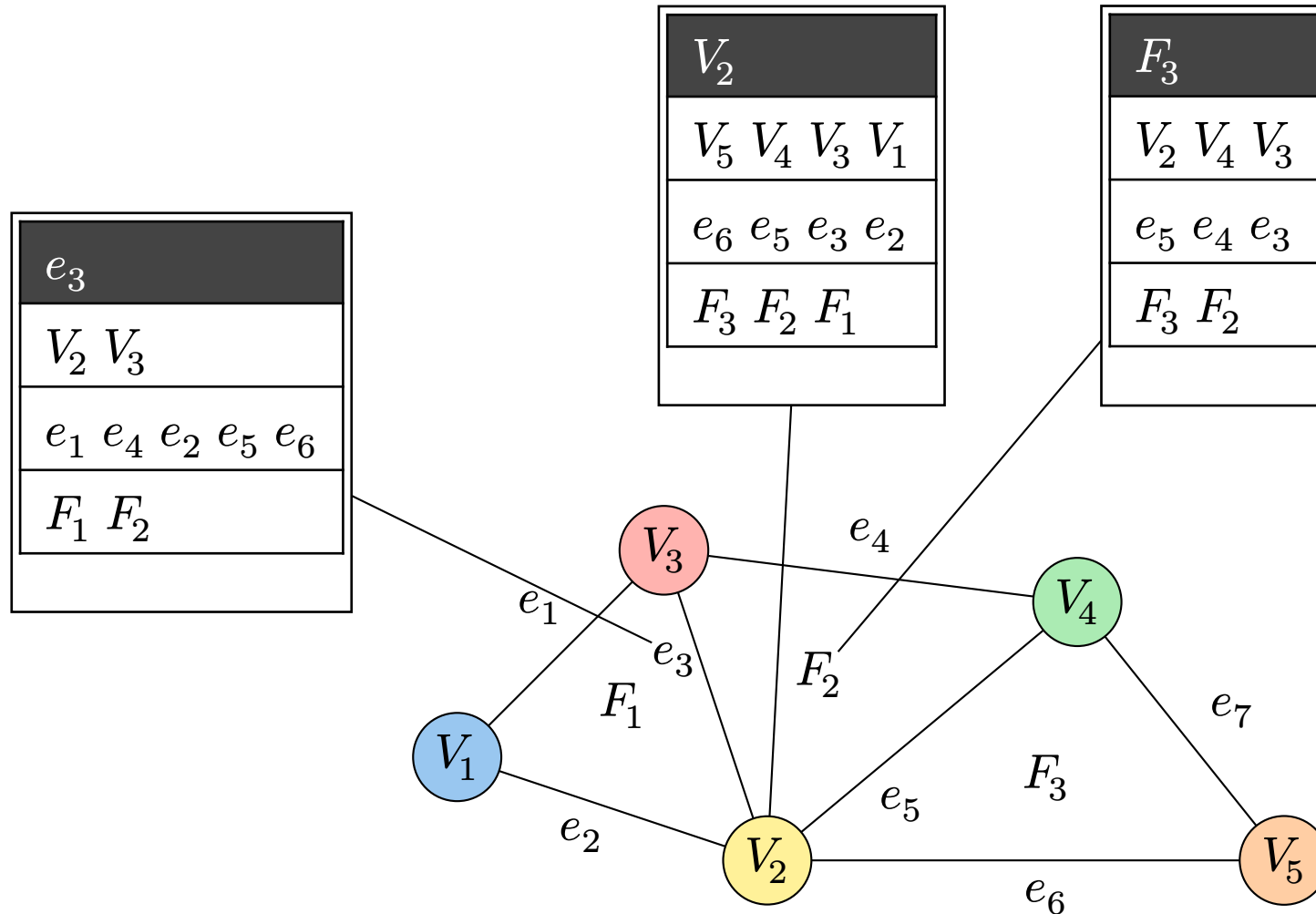
- Effizientes Traversieren der Topologie
- Zusätzlicher Speicher

**Winged Edge:** Erweiterte Mesh-Darstellung

- Speichert nur einige Adjazenzbeziehungen; kann auf Anfrage andere ableiten
- Wenig zusätzlicher Speicherplatz (feste Datensätze)

# Mesh Representation

## Adjacency Lists



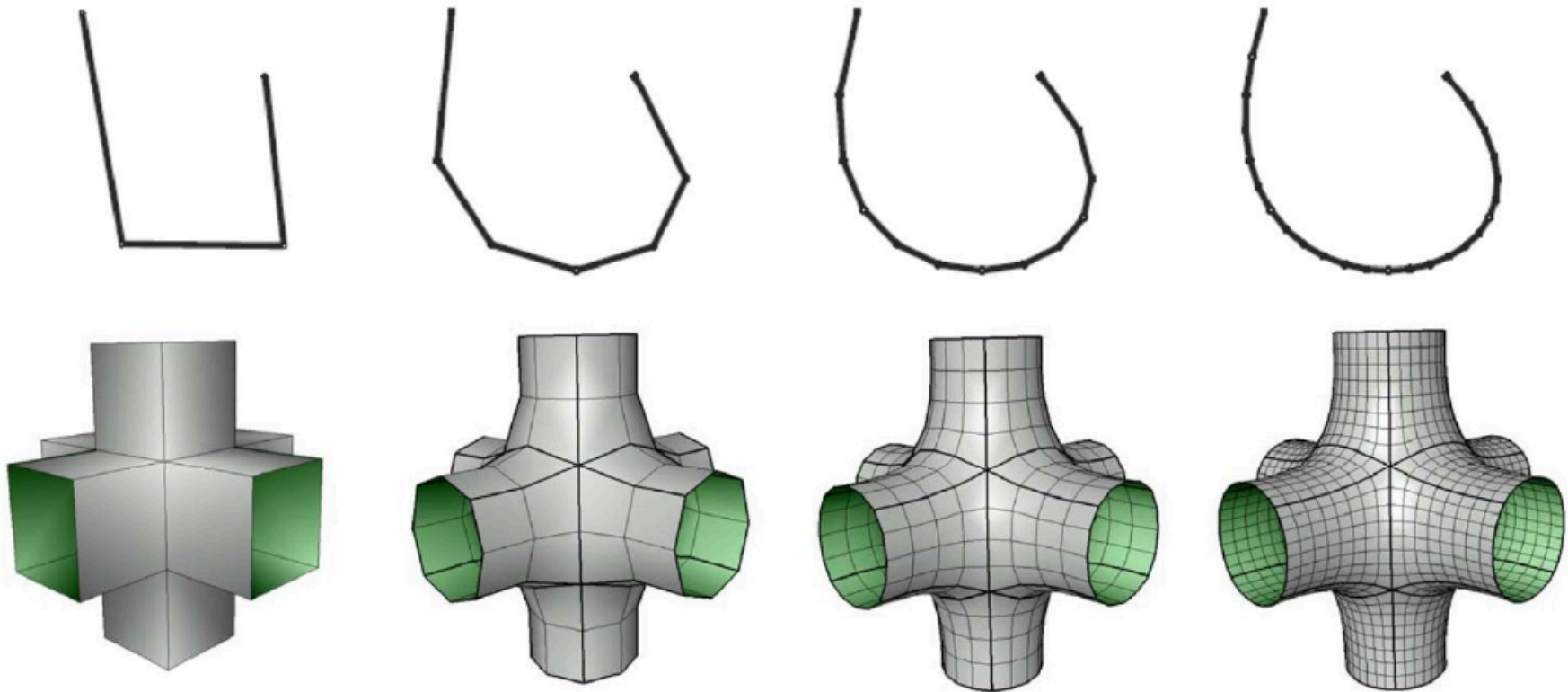
# Subdivision



# Subdivision

**Frage:** Wie macht man eine glatte Kurve?

**Antwort:** Schneiden der Ecken der Polyline so lange, bis Sie etwas erhalten, das keine Ecken mehr hat also glatt bzw. smooth ist.



# Subdivision

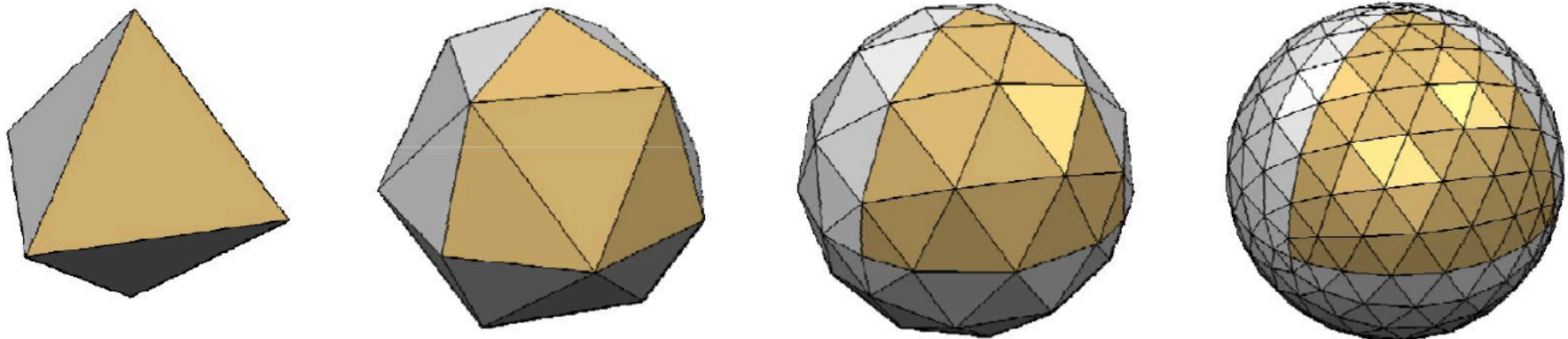
Eigenschaften:

- Genau
- Lokale Unterstützung von Problempunkten
- Affine Invariante
- Arbiträre Topologie
- Garantierte Kontinuität
- Effiziente Anzeige

# Subdivision

Grobmaschiges Mesh & Unterteilungsregel

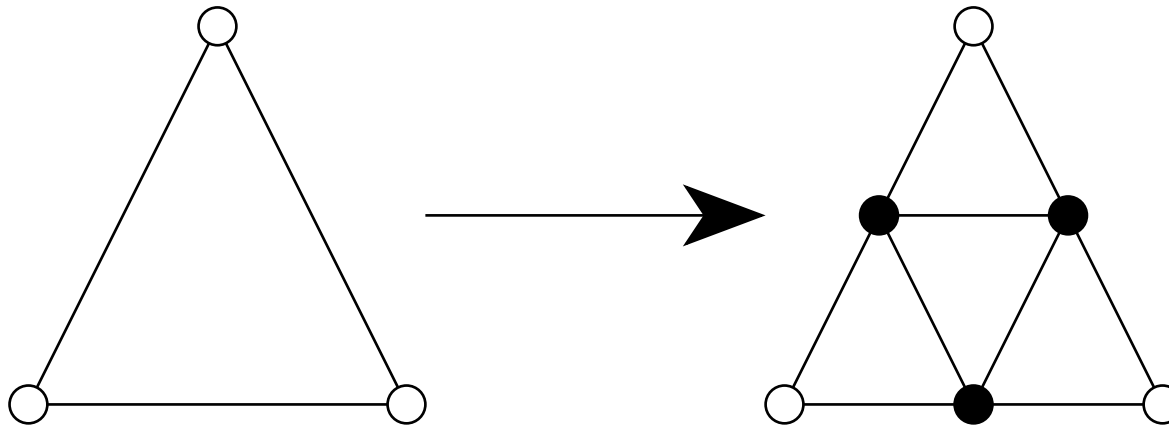
- Glatte Oberfläche als Grenze der Folge von Verfeinerungen definieren



# Subdivision

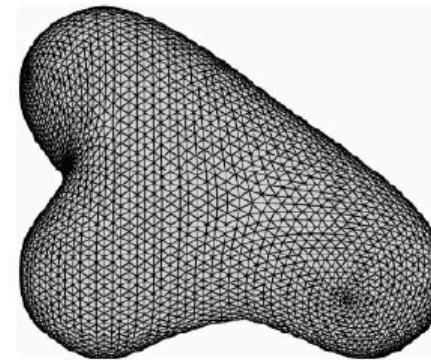
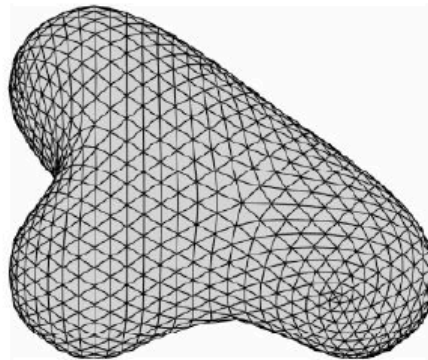
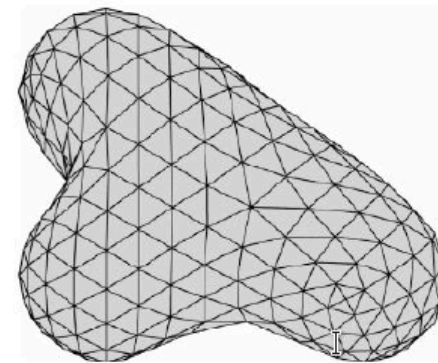
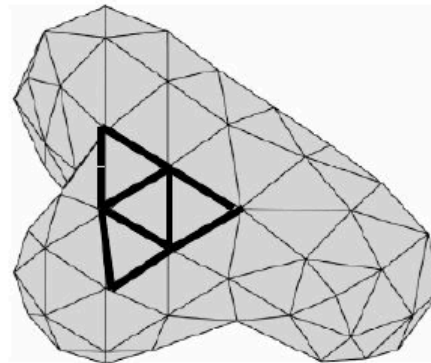
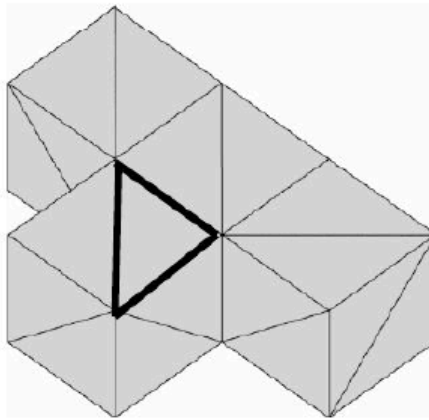
## Loop Unterteilungsregel

Jedes Dreieck in 4 Dreiecke verfeinern, indem jede Kante geteilt und neue Eckpunkte verbunden werden.



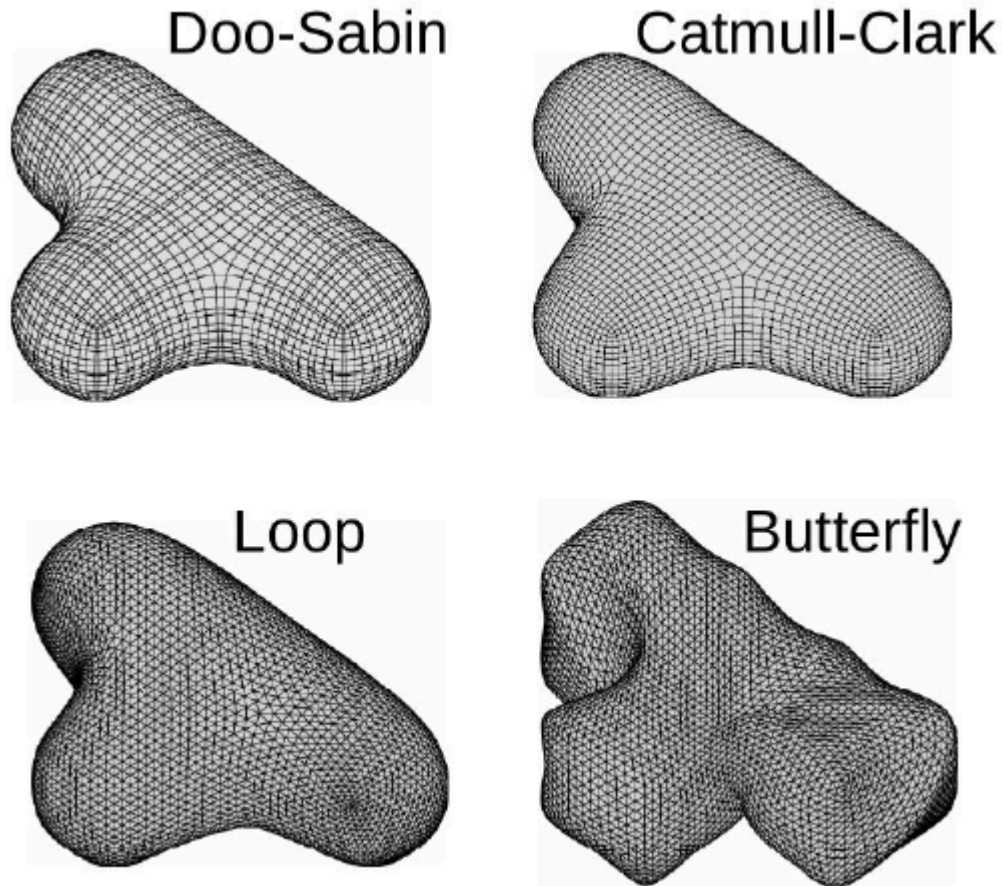
# Subdivision

## Loop Unterteilungsregel



# Subdivision

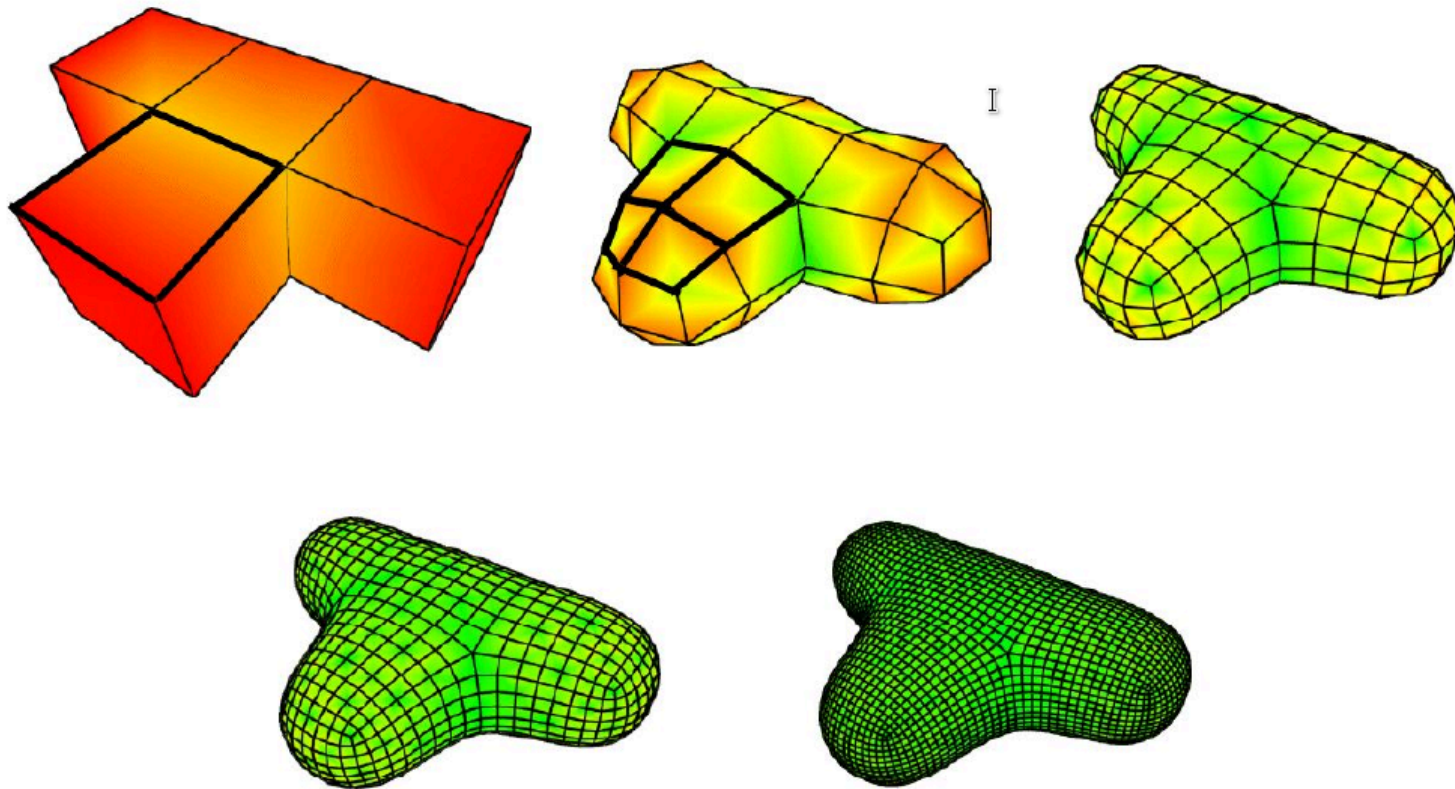
- Viele weitere Methoden möglich („Subdivision Zoo“)
- Blender verwendet vor allem den Catmull-Clark Algorithmus





# Subdivision

## Catmull-Clark



# Subdivision

Alle Unterteilungsschemata haben 2 Schritte:

1. **Aufspaltung oder Verfeinerungsschritt:** Topologische Regel, die neue Eckpunkte einführt und die Konnektivität modifiziert
2. **Schritt der Mittelwertbildung oder Glättung:** Geometrische Regel, die die Positionen für neue oder alle Scheitelpunkte durch gewichtete Mittelwerte berechnet.



# Adaptive Subdivision

Adaptive Approximation: Oberfläche wird nur dort unterteilt, wo die Topologie der Oberfläche mehr Details erfordert

- Auswahl der kritischen Netzbereiche (Scheitelpunkte) von Hand
- oder automatische Erkennung von gekrümmten (weiter unterteilen) vs. flachen (nicht weiter unterteilen) Bereichen

# Polygonal Meshes vs. Subdivision

- Erstellung einer Subdivision Surface durch Modifier
- Zwei Arten werden unterstützt:
  - HSDS Modifier bringt hierarchische Subdivision Surfaces
  - MeshSmooth Modifier bringt Weichzeichnung
- Modifier werden am besten als Finishing Tool genutzt

## HSDS

## Richtlinie 1

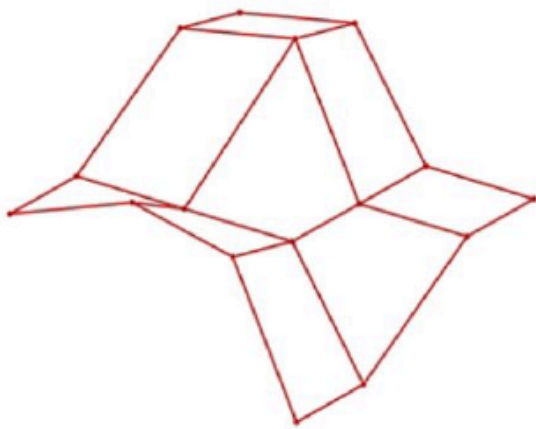
The HSDS modifier implements Hierarchical SubDivision Surfaces. It is intended primarily as a finishing tool rather than as a modeling tool. For best results, perform most of your modeling using low-polygon methods, and then use HSDS to add detail and adaptively refine the model.

Quelle: 3DS Max Help

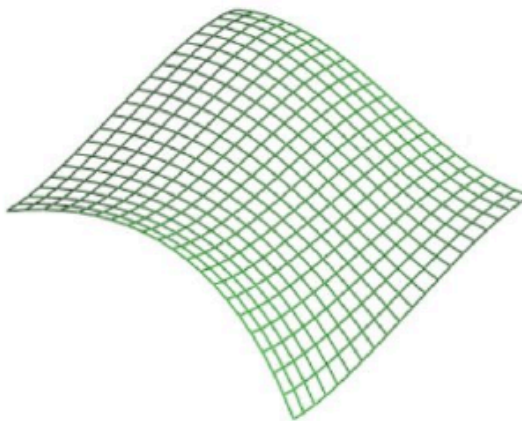
# Param. Oberflächen

# Gekrümmte Polygone

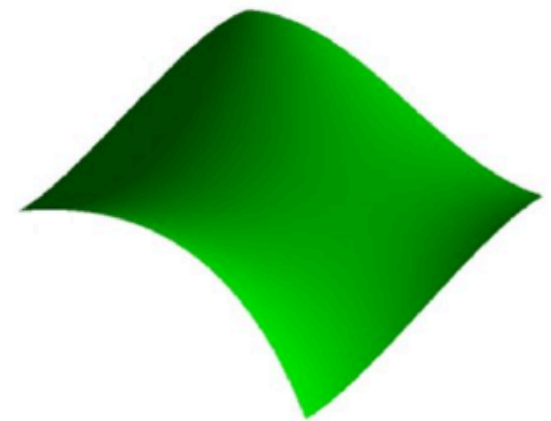
Rendering von gekrümmten Polygonen



(a)



(b)



(c)

(a) Gitter mit 16 Kontrollpunkten

(b) Bezierfläche aus 400 Quads im Drahtgittermodell

(c) Bezierfläche aus 400 Quads gefüllt und beleuchtet

# Parametrische Oberflächen

- Verallgemeinerung parametrischer Kurven
  - bi-polynomial, z.B. bikubische parametrische Flächen
  - stückweise parametrische Flächen
  - z.B. Bézier-Oberfläche, NURBS-Oberfläche, ...
- Häufig verwendet für Computer Aided Design (CAD)
- Mittlerweile Echtzeit-Renderings möglich
- Zukünftig wohl sehr häufige Anwendungsform

# Parametrische Oberflächen

Polygon Meshes  
Games

Subdivision Surfaces  
Filme

Parametrische Oberflächen  
CAD/CAM



erhöhte  
Genauigkeit

verlangsamtes  
Rendering

# Optimierung

# Polygon Mesh Optimierung

- Hauptnachteil von Polygonnetzen:
  - oft hohe Polygonanzahl zur Synthese eines Objekts für eine qualitativ hochwertige Edition
  - Heutige Modelle > 10 Mio Polygone möglich

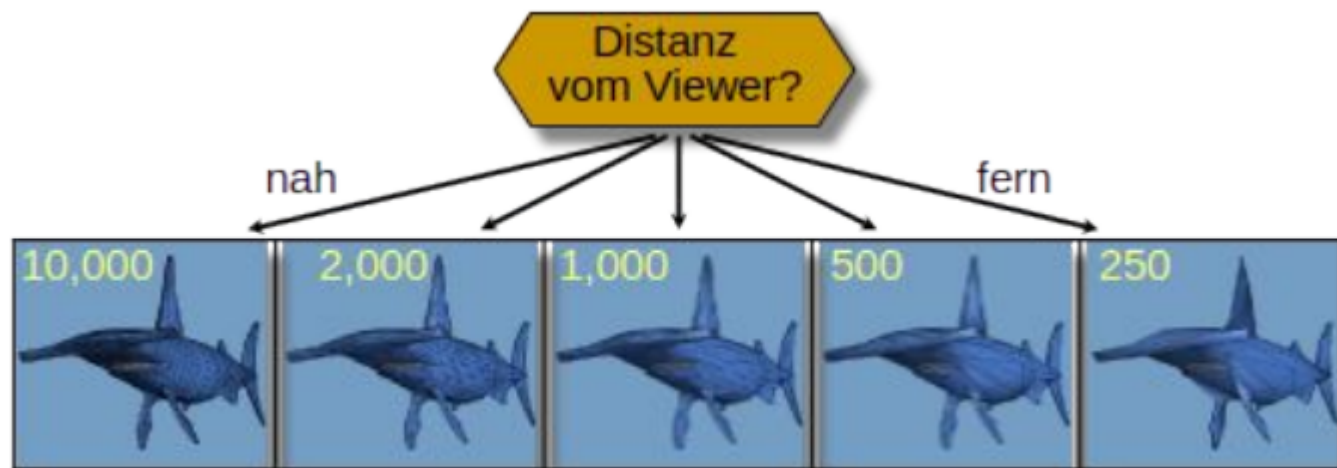
„It makes no sense to use 500 polygons in describing an object if it covers only 20 raster units on the display ... For example, when we view the human body from a very large distance, we might need to present only specks for the eyes, or perhaps a block for the head, totally eliminating the eyes from consideration“

*(James H. Clark, 1976)*



# Polygon Mesh Optimierung

- Polygone auf ein Niveau reduzieren, das der geforderten Qualität angemessen ist
  - Anwendung: Grad der Detailannäherung
  - [Clark76, Funkhouser93]
  - VRML, OpenSceneGraph: LOD-Knoten
- Problem: „PopUp-Effekt“

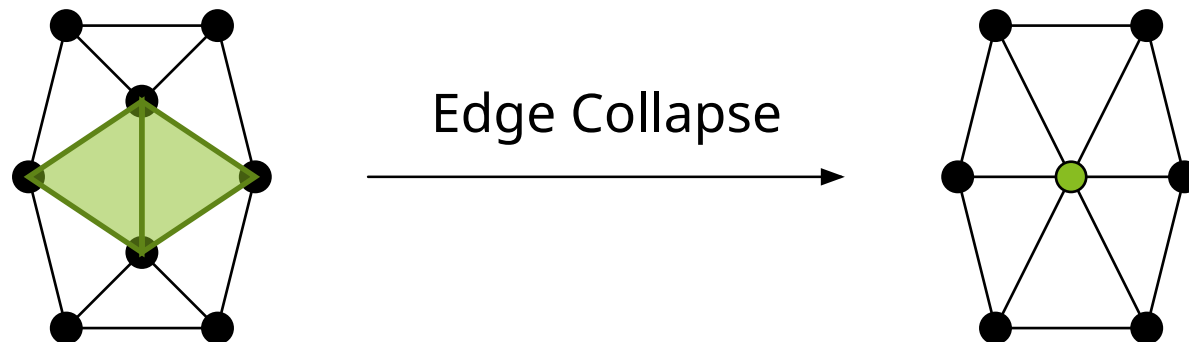


# Progressive Meshes

## Edge Collapse

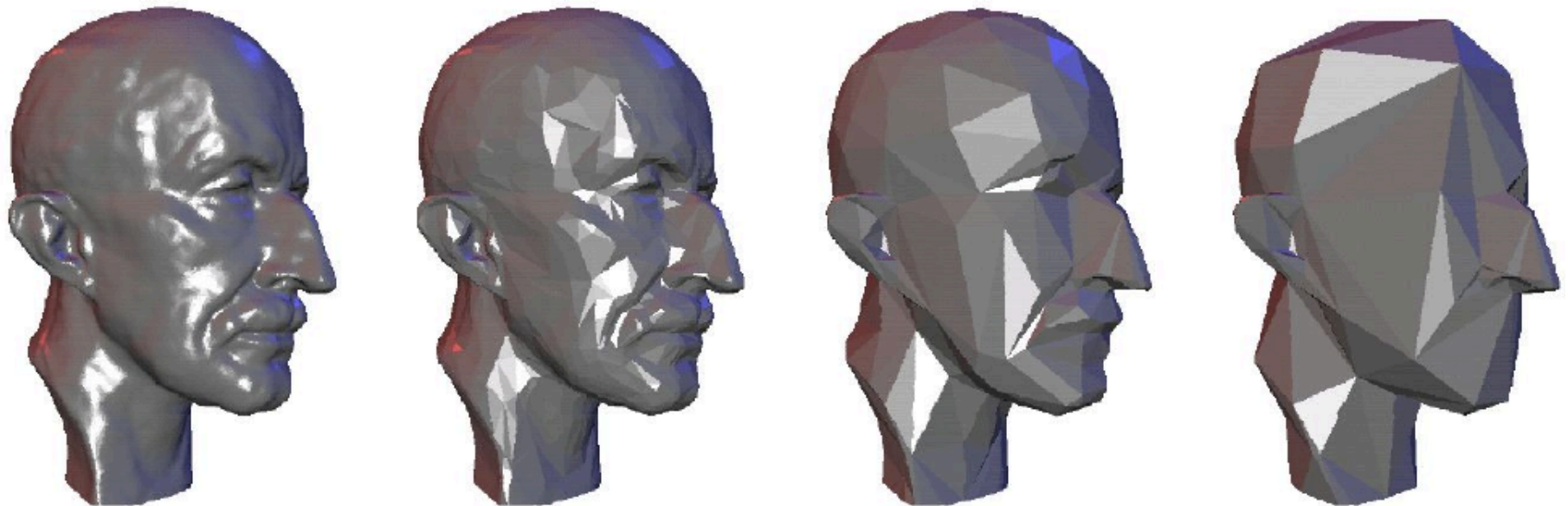
**Idee:** Sequenz von Kanteneinbrüchen (edge collapses) anwenden

- Erstellung einer Hierarchie von Objekten
- Blender: Decimate Modifier
- In modernen Engines (bspw. UE4) teilweise integriert



# Progressive Meshes

## Edge Collapse



# Progressive Meshes

## Edge Collapse & Vertes Split

### Welche Kanten sind auszuwählen

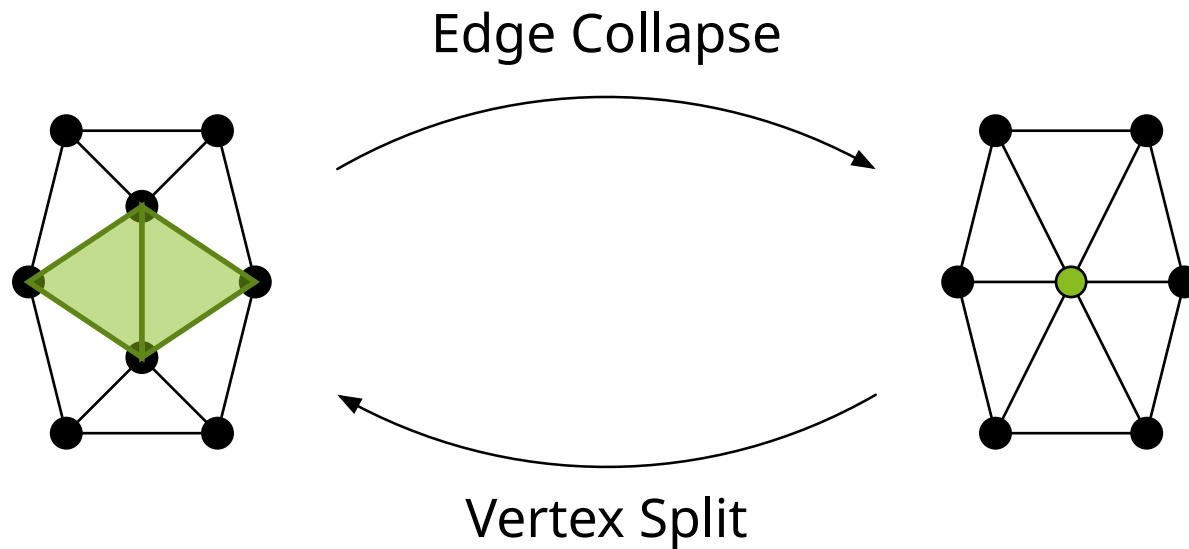
- einfache Heuristik: Kanten, die enge Polygone mit ähnlicher Orientierung verbinden
- oder komplexere Heuristiken

### Kantenkollaps ist umkehrbar!

- inverse Operation: Vertex Split
- kann erforderliche Vertex-Informationen in einem gröberen Netz speichern
- verlustfrei

# Progressive Meshes

## Edge Collapse & Vertex Split



# Progressive Meshes

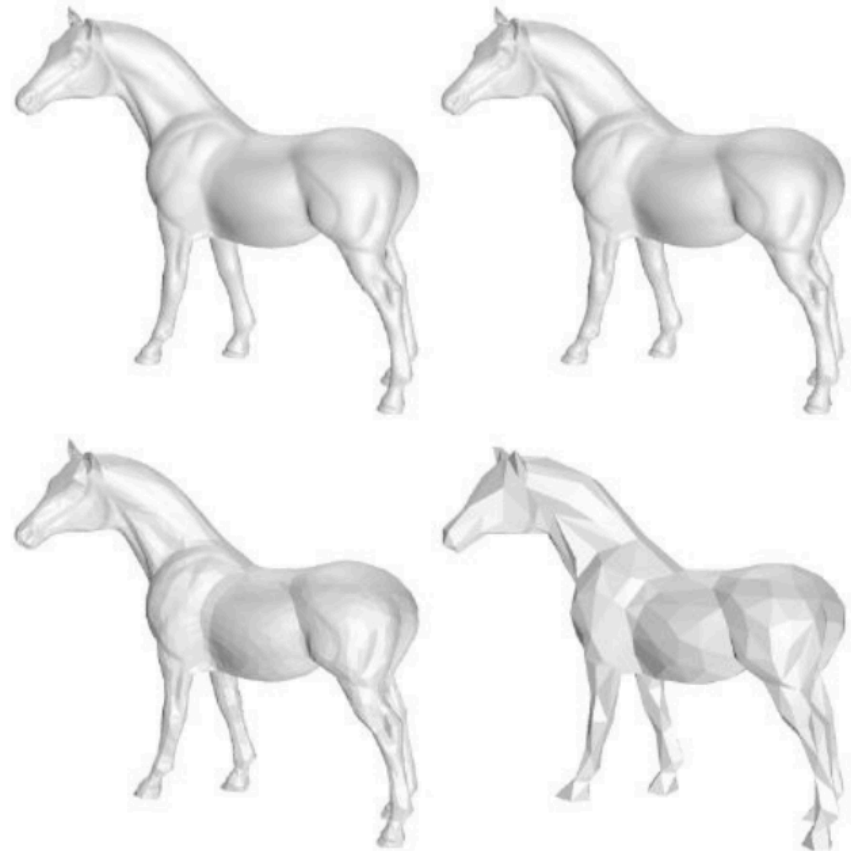
## Anwendung:

- continuous level of detail
- progressive Übertragung / Transmission
- selektive Verfeinerung, z.B. betrachterabhängig
- Mesh compression

# LOD

# Continuous Level of Detail (CLOD)

- Modell mit mehreren Auflösungen, das durch Vereinfachung des ursprünglichen Modells (in der linken oberen Ecke) erhalten wurde
- Anzahl der Dreiecke beträgt jeweils
  - 96966
  - 13334
  - 3334
  - 668

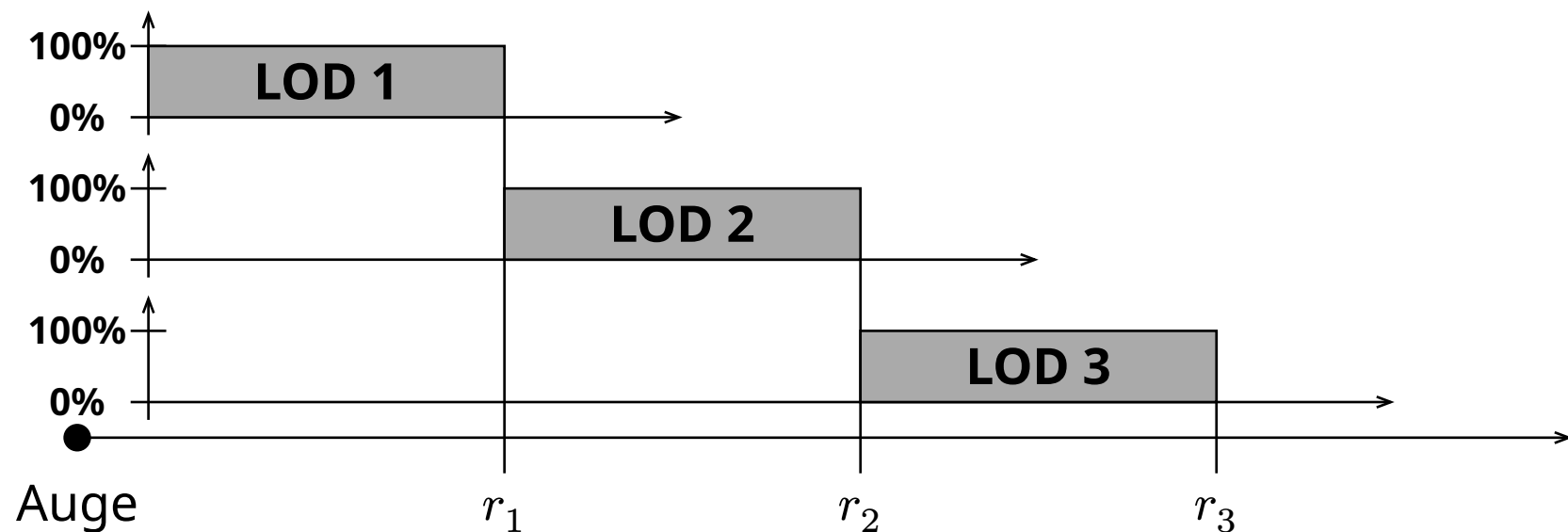




# Level of Detail

## Switch-LoD

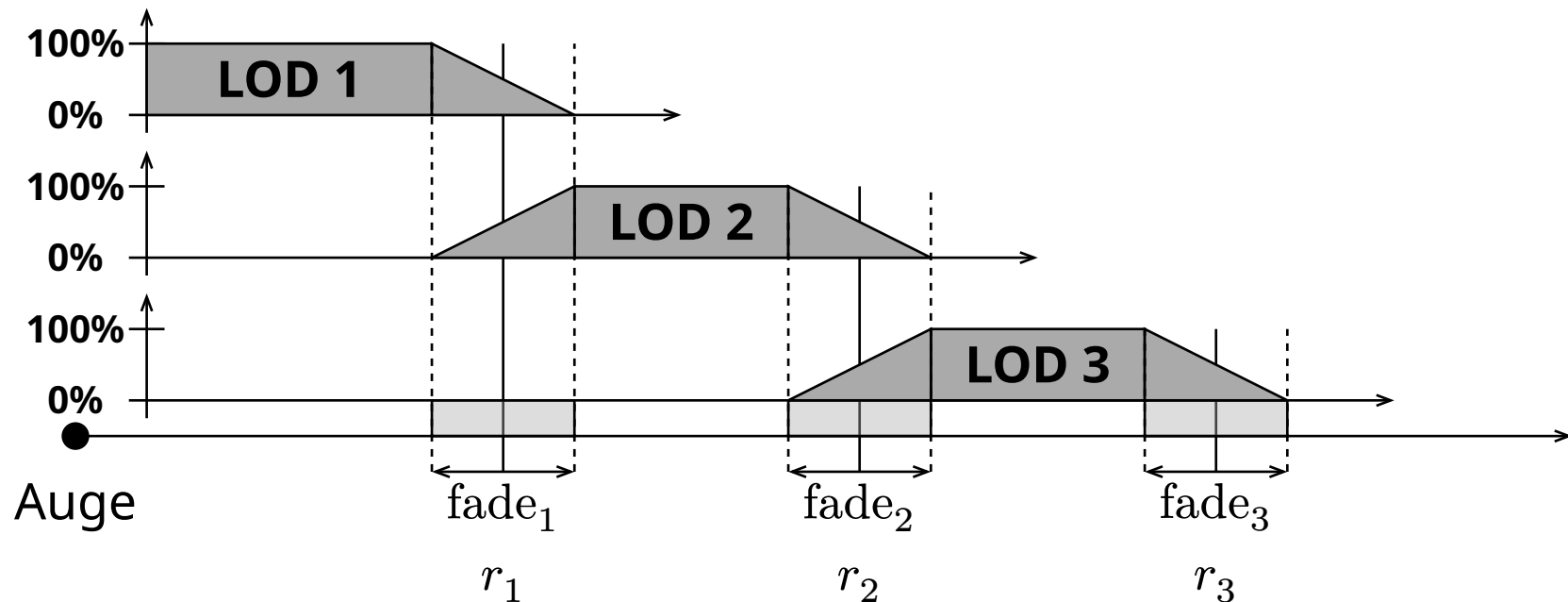
- Einfachste Form zur Anwendung von LoD-Stufen
- Harte Abstandsgrößen definieren Anzeige der Detailstufen



# Level of Detail

## Fade-LoD

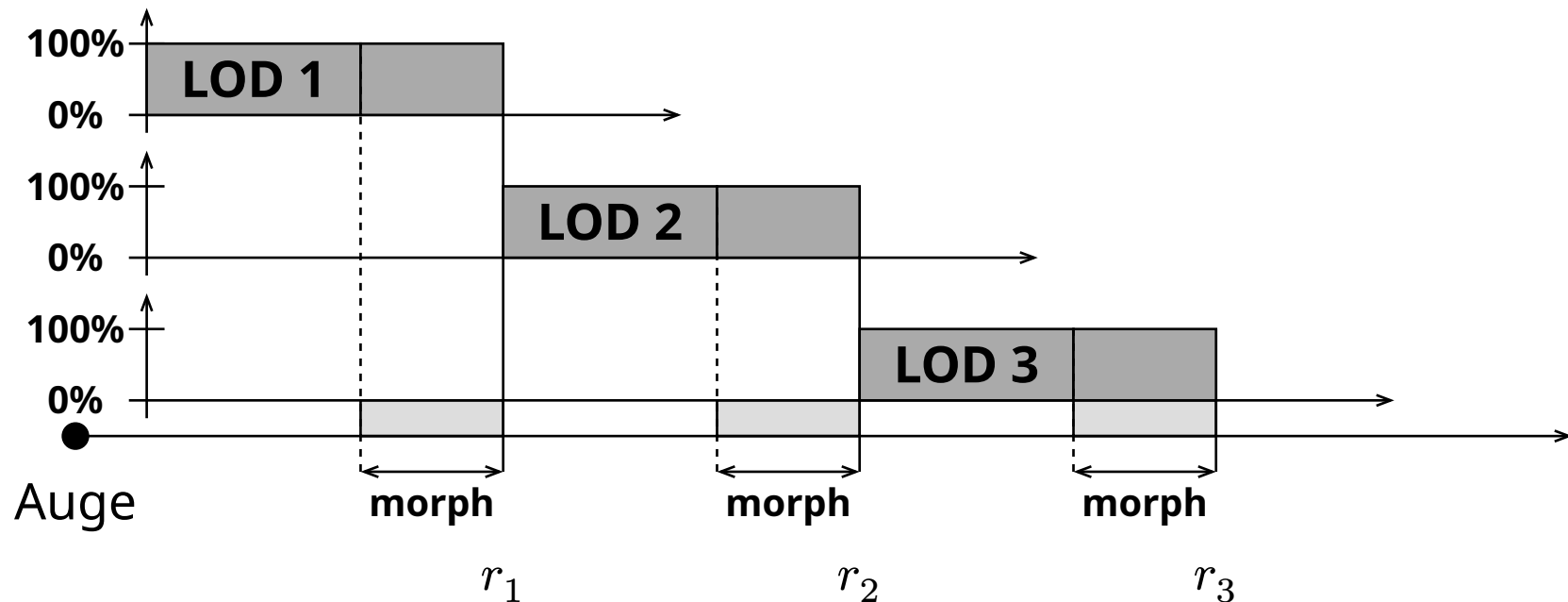
- “Überblendtechnik”, um LoD-Stufen optisch zu verschmelzen
- Verschmelzung durch Farb-Mischung / alpha blending



# Level of Detail

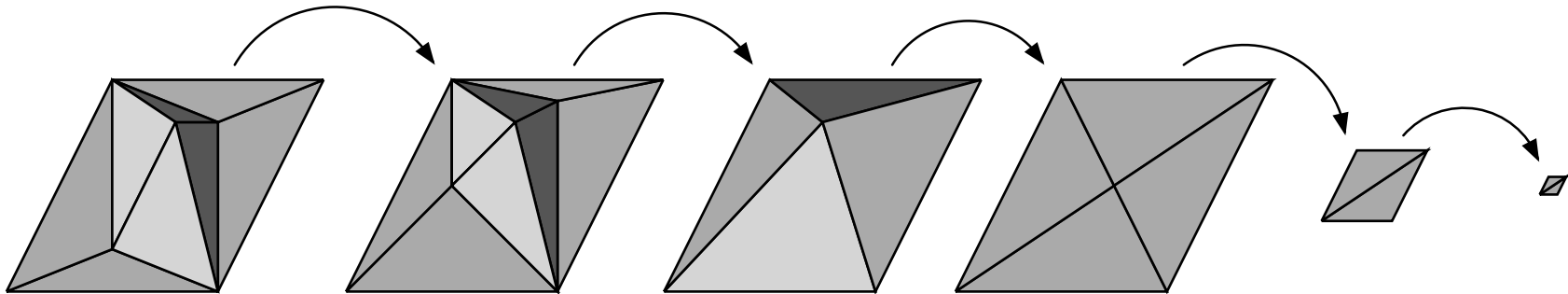
## Morph-LoD

- Kontinuierliche Formveränderung zwischen LoD-Stufen
- Bei komplexen Modellen erheblicher Rechenaufwand → Vertex Shader



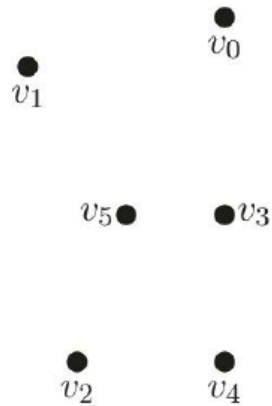
# Level of Detail

## Morph-LoD

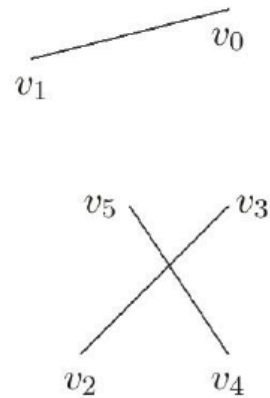


# Grafikprimitive

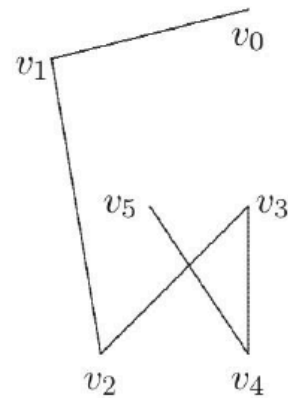
# Grafikprimitive



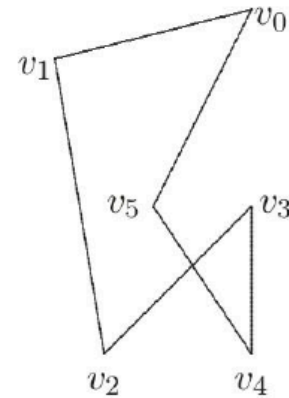
GL\_POINTS



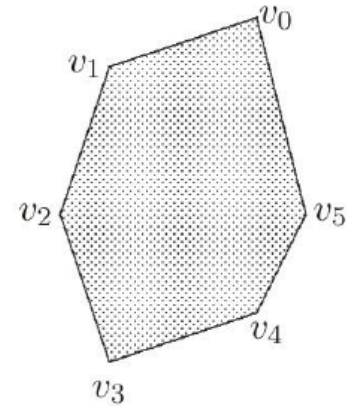
GL\_LINES



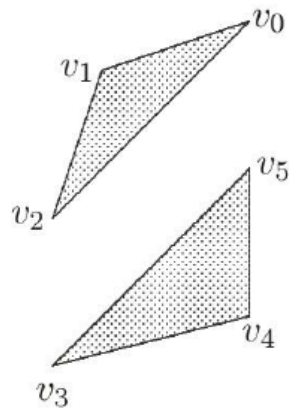
GL\_LINE\_STRIP



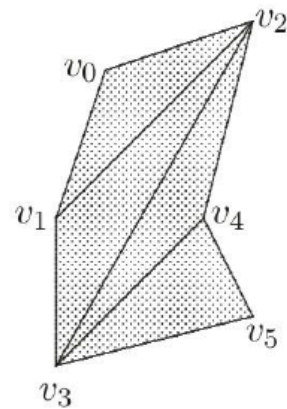
GL\_LINE\_LOOP



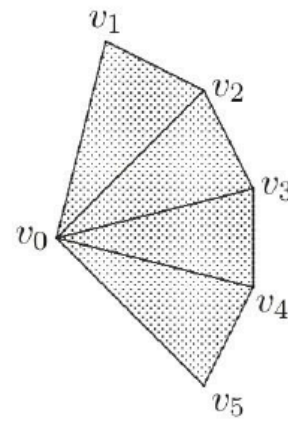
GL\_POLYGON



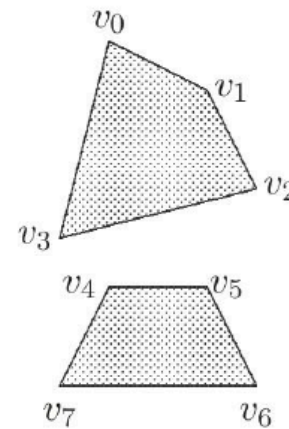
GL\_TRIANGLES



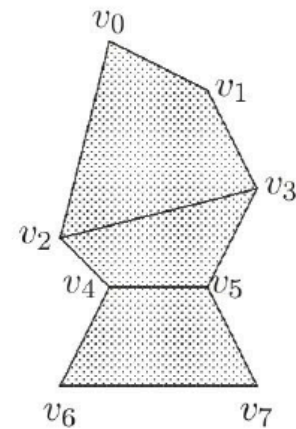
GL\_TRIANGLE\_STRIP



GL\_TRIANGLE\_FAN



GL\_QUADS



GL\_QUAD\_STRIP

# Grafikprimitive

- OpenGL Compatibility Profile (10 Primitive) vs. Core Profile (7 Primitive)
- Vulkan reduziert auf 6 Primitive, zusätzliche Primitive für spezielle Shader möglich (bspw. \_LINE LIST WITH ADJACENCY)

| Grafik-Primitive in Vulkan           | Grafik-Primitive in OpenGL |
|--------------------------------------|----------------------------|
| VK PRIMITIVE TOPOLOGY POINT LIST     | GL POINTS                  |
| VK PRIMITIVE TOPOLOGY LINE LIST      | GL LINES                   |
| VK PRIMITIVE TOPOLOGY LINE STRIP     | GL LINE STRIP              |
| VK PRIMITIVE TOPOLOGY TRIANGLE LIST  | GL TRIANGLES               |
| VK PRIMITIVE TOPOLOGY TRIANGLE STRIP | GL TRIANGLES STRIP         |
| VK PRIMITIVE TOPOLOGY TRIANGLE FAN   | GL TRIANGLES FAN           |

# Grafikprimitive

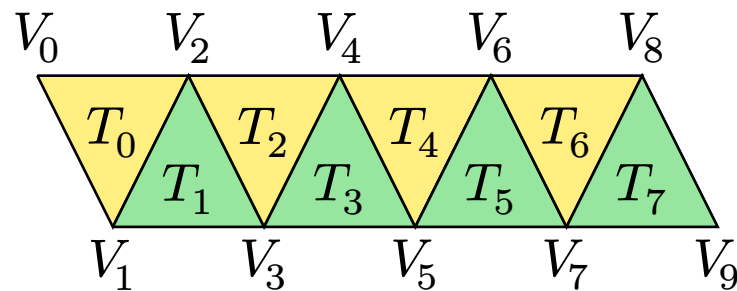
---

## **Triangle Strips**



# Triangle Strips

- Ohne strips: 8 triangles \* 3 vertices = 24 vertices
- Mit strips: 1 vertex per triangle anstelle von 3



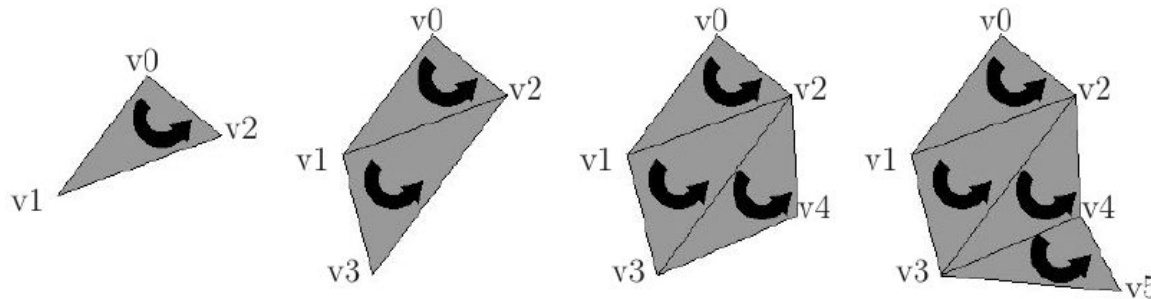
- Was wir an die Graphic-Hardware senden:
  - Startkosten:  $v_0, v_1$
  - dann  $v_2$  ( $T_0$ ),  $v_3$  ( $T_1$ ),  $v_4$  ( $T_2$ ),  $v_5$  ( $T_3$ ),  $v_6$  ( $T_4$ ),  $v_7$  ( $T_5$ ),  $v_8$  ( $T_6$ ),  $v_9$  ( $T_7$ )

# Triangle Strips

- 10 Vertices statt 24
  - $\frac{100 \times 10}{24} = 41.7\%$  der Daten
  - $\frac{10}{8} = 1.25$  Vertices per Triangle
- **Ergebnis:** wir können Geometriephase mehr als 2x schneller erwarten
- Definition eines Dreieckstreifens bewirkt eine Änderung der Orientierung zwischen benachbarten Dreiecken im Streifen
  - Intern wird die Reihenfolge gegen den Uhrzeigersinn konsistent gehalten, indem die Eckpunkte 0-1-2, 1-3-2, 2-3-4, 3-5-4, ... überquert werden.

# Triangle Strips

## OpenGL Implementation

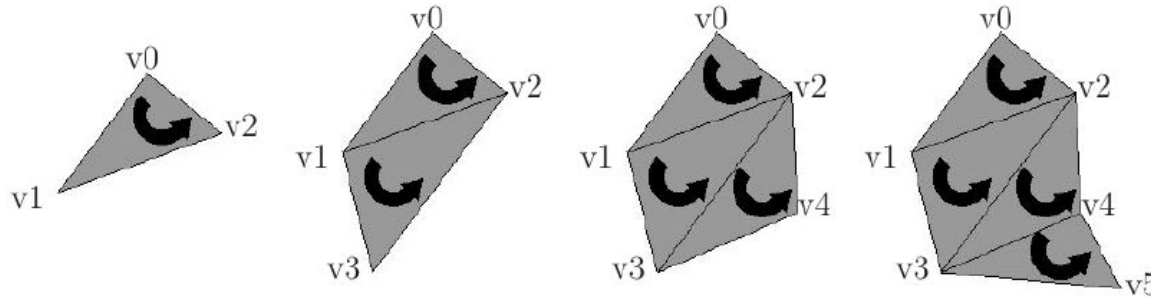


### Compatibility Profile

GLSL

```
1  glBegin(GL_TRIANGLE_STRIP)
2      glVertex3fv(v0);
3      glVertex3fv(v1);
4      glVertex3fv(v2);
5      glVertex3fv(v3);
6      glVertex3fv(v4);
7      glVertex3fv(v5);
8  glEnd();
```

# Triangle Strips



## Core Profile

GLSL

```
1  GLBatch triangleBatch;
2  GLShaderManager SM;
3  float grey[] = {0.5, 0.5, 0.5, 1.0};
4  float verts[][3] = {v0, v1, v2, v3, v4, v5};
5  triangleBatch.Begin(GL_TRIANGLE_STRIP, 6);
6  triangleBatch.CopyVertexData3f(verts);
7  triangleBatch.End();
8  SM.UseStockShader(GLT_SHADER_IDENTITY, grey);
9  triangleBatch.Draw();
```

# Triangle Strips

## Swaps/Touch

- Durchführen eines Swaps / Tausch für T3
- Startkosten: v0, v1 dann
  - v2 (T0)
  - v3 (T1)
  - v2
  - v4 (T2)
  - v5 (T3)
  - v6 (T4)
- Degeneriertes Dreieck (0-Fläche): v2, v3, v2